Banco de Dados — Rodada 2

Oficina de questões comentadas

Tópicos Especiais II — ENADE · SI · 2026/2

Prof. M.Sc. Howard Cruz Roatti

Hoje é treino, não revisão

A teoria você já viu (12/08)

- Modelo relacional → SQL → transações → NoSQL.
- Hoje **não** repetimos a recaptura.

O que muda

- **12 questões novas**, nenhuma repetida.
- Foco no **raciocínio de resolução**.

Como vamos jogar

1. Leia a situação-problema.
2. **Responda** (mentalmente ou no papel).
3. Só então viramos o **gabarito comentado**.
4. Vale mais **errar e entender** do que acertar no chute.

> 💡 O ENADE cobra **análise**: leia o contexto, elimine distratores, decida. A pressa é a maior armadilha.

O mapa da rodada de hoje

- **Q1** Álgebra — diferença
- **Q2** SQL — o que nunca ocorreu
- **Q3** SQL — WHERE × HAVING
- **Q4** Modelagem — atributo de relacionamento
- **Q5** Normalização — 3FN
- **Q6** Integridade referencial
- **Q7** Transações — ACID
- **Q8** Isolamento — leitura fantasma
- **Q9** Índices — quando o índice "morre"
- **Q10** NoSQL — qual modelo?
- **Q11** Views — abstração e segurança
- **Q12** SQL — ordem lógica de execução

📌 Mesmos **eixos** do ENADE de BD, **situações diferentes** das que já resolvemos.

🔥 Rodada 2

12 questões · responda antes de virar o slide!

Q1 — Álgebra relacional

Uma empresa mantém `Cliente(id, nome)` e `Pedido(id, id_cliente, valor)`. O time de marketing quer os **nomes dos clientes que nunca fizeram um pedido**.

Assinale a expressão de álgebra relacional correta.

A) $\pi_{nome}(\text{ Cliente } \bowtie \text{ Pedido })$
B) $\pi_{nome}(\text{ Cliente }) - \pi_{nome}(\text{ Cliente } \bowtie \text{ Pedido })$
C) $\sigma_{id_cliente = NULL}(\text{ Cliente } \bowtie \text{ Pedido })$
D) $\pi_{nome}(\text{ Cliente }) \cup \pi_{nome}(\text{ Pedido })$

Q1 — Gabarito: B

Como pensar: "quem **nunca** fez X" = *todos* **menos** *quem fez* → operador de **diferença (−)**.

- π_{nome}(Cliente) = todos os nomes; π_{nome}(Cliente ⋈ Pedido) = quem **tem** pedido. A diferença sobra **quem não tem**. ✅
- **A** traz só quem **tem** pedido; **C** usa `NULL` em condição de junção (a junção já descartou esses clientes); **D** mistura relações incompatíveis.

> 💡 Em SQL isto vira `LEFT JOIN … WHERE id_cliente IS NULL` ou `NOT EXISTS`.

Q2 — SQL: o que nunca ocorreu

Em `Produto(id, nome)` e `ItemVenda(id_produto, qtd)`, a equipe quer os **produtos que nunca foram vendidos**.

Qual consulta atende?

A) `SELECT nome FROM Produto p JOIN ItemVenda i ON i.id_produto = p.id`

B) `SELECT nome FROM Produto WHERE id NOT IN (SELECT id_produto FROM ItemVenda)`

C) `SELECT nome FROM Produto WHERE qtd = 0`

D) `SELECT nome FROM Produto GROUP BY nome HAVING SUM(qtd) = 0`

Q2 — Gabarito: B

Como pensar: "nunca vendido" = produto **ausente** de `ItemVenda` → **`NOT IN`** / **`NOT EXISTS`** / `LEFT JOIN … IS NULL`.

- **A** (`JOIN` interno) devolve justamente os **que venderam** — o oposto. ✅ = **B**.
- **C** usa `qtd` (coluna de `ItemVenda`) na tabela `Produto`; **D** só encontraria quem **está** em `ItemVenda` com soma zero, não os ausentes.

> ⚠️ Cuidado com `NOT IN` e valores **NULL** na subconsulta — na prática, prefira `NOT EXISTS`.

Q3 — SQL: WHERE × HAVING

Deseja-se a **média de preço por categoria**, considerando **apenas produtos ativos**, exibindo só as categorias cuja **média passa de R$ 100**.

Qual combinação está correta?

A) `WHERE ativo = 'S'` … `GROUP BY categoria` … `HAVING AVG(preco) > 100`

B) `WHERE ativo = 'S' AND AVG(preco) > 100` … `GROUP BY categoria`

C) `HAVING ativo = 'S'` … `GROUP BY categoria`

D) `GROUP BY categoria` … `WHERE AVG(preco) > 100`

Q3 — Gabarito: A

Como pensar: dois filtros de naturezas **diferentes**:

- Filtrar **linhas** (produto ativo) → `WHERE` (antes de agrupar).
- Filtrar **grupos** por um **agregado** (média) → `HAVING` (depois de agrupar). ✅
- **B** e **D** usam `AVG` no `WHERE` (inválido — o agregado ainda não existe); **C** usa `HAVING` para uma condição de **linha**, que pertence ao `WHERE`.

Q4 — Modelagem: atributo de relacionamento

Um aluno cursa muitas disciplinas e uma disciplina tem muitos alunos (**N:M**). Cada matrícula guarda uma **nota**. Onde a `nota` deve ficar no modelo lógico?

A) Em `Aluno`, como coluna.

B) Em `Disciplina`, como coluna.

C) Na **tabela associativa** `Aluno_Disciplina(id_aluno, id_disciplina, nota)`.

D) Numa lista de notas separada por vírgula em `Aluno`.

Q4 — Gabarito: C

Como pensar: `nota` **não** pertence ao aluno (varia por disciplina) nem à disciplina (varia por aluno) — pertence ao **encontro** dos dois.

- Todo relacionamento **N:M** vira **tabela associativa**; atributos do relacionamento moram **nela**. ✅
- **A/B** duplicariam ou perderiam informação; **D** fere a **1FN** (atributo multivalorado).

Q5 — Normalização (3FN)

`Pedido(id_pedido, id_cliente, cidade_cliente)`, chave `id_pedido`, com a regra **`id_cliente → cidade_cliente`**. A cidade se repete em todo pedido do mesmo cliente.

A relação **viola** a:

A) 1FN, por atributo multivalorado.
B) 2FN, por dependência parcial.
C) 3FN, por **dependência transitiva**.
D) Nenhuma; já está na BCNF.

Q5 — Gabarito: C

Como pensar: `id_pedido → id_cliente → cidade_cliente`. A `cidade` depende da chave **através de** um atributo **não-chave** (`id_cliente`) → **dependência transitiva** → fere a **3FN**. ✅

- Não é **2FN**: a chave (`id_pedido`) é **simples**, então não há dependência **parcial**.
- **Correção:** `Cliente(id_cliente, cidade_cliente)` + `Pedido(id_pedido, id_cliente)`.

Q6 — Integridade referencial

Em `Cliente(id)` e `Pedido(id, id_cliente)` com FK `id_cliente → Cliente(id)`, tenta-se **excluir um cliente que possui pedidos**, e a FK está definida como `ON DELETE RESTRICT`.

O que acontece?

A) O cliente é apagado e os pedidos ficam com `id_cliente` órfão.
B) O SGBD **recusa** a exclusão para preservar a integridade referencial.
C) Os pedidos são apagados automaticamente junto.
D) O `id_cliente` dos pedidos vira `NULL`.

Q6 — Gabarito: B

Como pensar: a FK impede **referência órfã**. A ação escolhida decide o comportamento na exclusão do "pai":

- `RESTRICT` / `NO ACTION` → **bloqueia** enquanto houver filhos. ✅
- `CASCADE` → apagaria os pedidos junto (opção C); `SET NULL` → zeraria a FK (opção D). **A** violaria a integridade — o SGBD nunca permite.

> 💡 A ação é uma **decisão de projeto**: escolha conforme a regra de negócio, não por acaso.

Q7 — Transações (ACID)

Uma transferência **debita** de uma conta e **credita** em outra. Falta energia **após o débito** e **antes do crédito**. Ao reiniciar, nenhuma das duas alterações aparece.

Qual propriedade ACID garante esse comportamento?

A) **Atomicidade** — tudo ou nada.
B) Consistência — respeita as regras do esquema.
C) Isolamento — transações não se interferem.
D) Durabilidade — o que foi confirmado persiste.

Q7 — Gabarito: A

Como pensar: débito **sem** crédito é um estado **pela metade**. Como a transação **não** confirmou, o SGBD desfaz o débito no *recovery* → **tudo ou nada** = **Atomicidade**. ✅

- **Consistência** é a *garantia global* que a atomicidade ajuda a manter; **Isolamento** trata de concorrência; **Durabilidade** só valeria **após** um `commit` — que aqui não houve.

Q8 — Isolamento (leitura fantasma)

`T1` executa `SELECT COUNT(*) FROM Pedido WHERE valor > 1000` e obtém **10**. Antes de `T1` terminar, `T2` **insere e confirma** um novo pedido de R$ 1.500. `T1` **repete** a mesma consulta e obtém **11**.

A anomalia e o nível que a evita são:

A) Leitura suja — `READ COMMITTED`.
B) Leitura não repetível — `REPEATABLE READ`.
C) **Leitura fantasma** — `SERIALIZABLE`.
D) Perda de atualização — `READ UNCOMMITTED`.

Q8 — Gabarito: C

Como pensar: o que mudou entre as leituras **não** foi uma linha já lida, e sim o **surgimento de novas linhas** que passam a satisfazer o filtro → **leitura fantasma** (*phantom*). ✅

- `REPEATABLE READ` protege as **linhas já lidas**, mas **não** impede *inserções*; só `SERIALIZABLE` elimina o fantasma.
- *Dirty read* = ler dado **não confirmado** (não é o caso; `T2` deu `commit`).

Q9 — Índices: quando o índice "morre"

Existe um índice B-tree em `Cliente(data_cadastro)`. Qual consulta **tende a NÃO usar** esse índice?

A) `WHERE data_cadastro = DATE '2026-09-16'`

B) `WHERE data_cadastro BETWEEN :ini AND :fim`

C) `WHERE YEAR(data_cadastro) = 2026`

D) `WHERE data_cadastro > :corte ORDER BY data_cadastro`

Q9 — Gabarito: C

Como pensar: o índice guarda o valor **da coluna**, não o de `YEAR(coluna)`. Ao **aplicar uma função na coluna filtrada**, o SGBD não consegue navegar a árvore → **varredura completa** (predicado *não-sargable*). ✅

- **A/B/D** comparam a **própria coluna** (igualdade, faixa, ordenação) — casos clássicos em que a B-tree ajuda.
- **Correção:** reescrever como faixa — `data_cadastro >= '2026-01-01' AND data_cadastro < '2027-01-01'`.

Q10 — NoSQL: qual modelo?

Uma rede social precisa responder rápido a perguntas como "*amigos dos amigos que curtem X*", percorrendo **muitas conexões** entre pessoas.

Qual modelo NoSQL é o mais adequado?

A) Chave-valor (ex.: Redis).
B) Documento (ex.: MongoDB).
C) Colunar / família de colunas (ex.: Cassandra).
D) **Grafo** (ex.: Neo4j).

Q10 — Gabarito: D

Como pensar: o problema é **relacionamento** entre entidades e **travessia** de muitos saltos. Bancos de **grafo** guardam vértices e arestas e percorrem conexões sem *joins* caros. ✅

- **Chave-valor**: busca por chave, sem consulta rica; **Documento**: agregados autocontidos; **Colunar**: escrita/leitura massiva por linhas largas. Nenhum é feito para *travessia* de rede.

> 💡 Regra prática: escolha o modelo pelo **padrão de acesso**, não pela moda.

Q11 — Views: abstração e segurança

Um DBA cria `VIEW Folha_Publica AS SELECT nome, cargo FROM Funcionario` (**sem** o salário) e concede acesso a essa view em vez da tabela.

Qual é o principal benefício obtido?

A) A view **acelera** a consulta por armazenar os dados fisicamente.
B) A view oferece **abstração e segurança**, expondo só as colunas permitidas.
C) A view **substitui** a necessidade de índices.
D) A view garante **isolamento** entre transações concorrentes.

Q11 — Gabarito: B

Como pensar: uma view comum é uma **consulta com nome** — não guarda dados; ela **filtra o que o usuário enxerga**. Dar acesso à view (e não à tabela) **esconde o salário**. ✅

- **A** confunde com **view materializada** (essa sim armazena resultado); **C** e **D** tratam de temas não relacionados (desempenho/concorrência).

Q12 — SQL: ordem lógica de execução

Por que o comando abaixo **falha** em muitos SGBDs?

```
SELECT preco * qtd AS total FROM Item WHERE total > 500;
```

A) Falta `GROUP BY`.

B) `WHERE` é avaliado **antes** do `SELECT`, então o *alias* `total` ainda **não existe**.

C) Não se pode multiplicar colunas em `SELECT`.

D) `total` é palavra reservada.

Q12 — Gabarito: B

Como pensar: a ordem **lógica** é `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY`. O `WHERE` roda **antes** do `SELECT`, onde o *alias* nasce. ✅

- **Correções:** repetir a expressão (`WHERE preco*qtd > 500`) ou envolver numa **subconsulta**/CTE e filtrar por `total` na consulta externa.
- O `ORDER BY`, por rodar **depois** do `SELECT`, **aceita** o *alias* `total`.

Fim da Rodada 2 🗄️

Próximo: Aulão de BD (23/09) — eixo agregação, em grupos.

Gabarito: 1B · 2B · 3A · 4C · 5C · 6B · 7A · 8C · 9C · 10D · 11B · 12B

> 💡 Errou alguma? Anote o **eixo** (não a questão) e revise a unidade correspondente antes do Simulado.